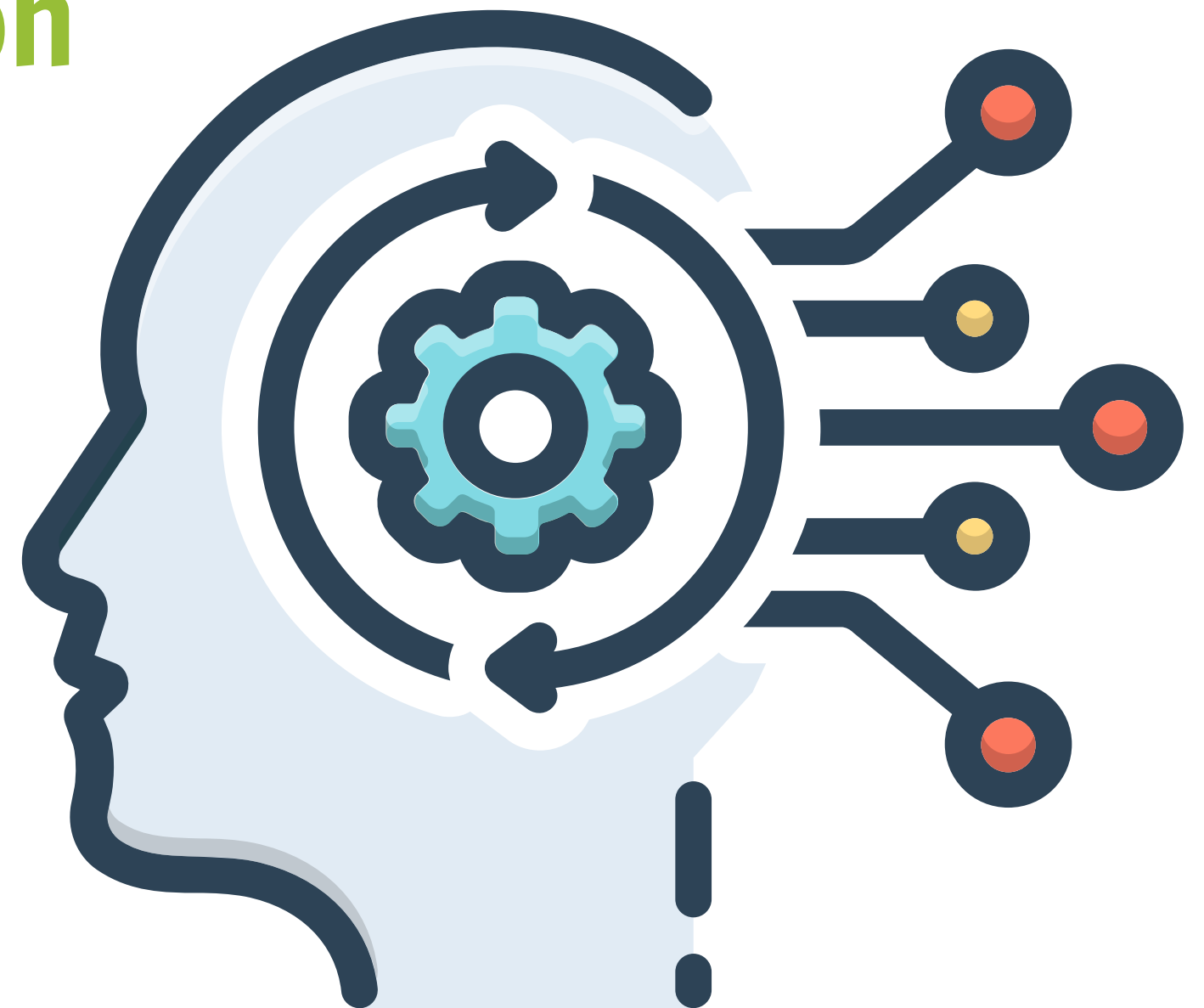


Conteneurisation & orchestration

Dockerfile

INFO – AGROTIC

Last update: oct. 2025



Qu'est ce qu'un Dockerfile ?

Un Dockerfile est un script texte qui contient des instructions ordonnées pour assembler une image Docker.

- Règles de syntaxe :
- Instructions en MAJUSCULES (convention)
- Commentaires commencent par #
- Chaque instruction crée une nouvelle couche
- L'ordre des instructions est important

Commandes sur Dockerfile

Méthode 3 : Via Dockerfile

Principe

- Lors de la construction d'une image personnalisée, vous pouvez intégrer des commandes directement dans le **Dockerfile**.
- Ces commandes seront exécutées lors de la construction de l'image, pas au démarrage du conteneur.

Commandes sur Dockerfile : FROM

Méthode 3 : Via Dockerfile

FROM – Définit l'image de base (doit être la première instruction)

dockerfile

```
FROM ubuntu:latest
```

Exécuter une commande simple

```
RUN echo "GeeksForGeeks"
```

Mettre à jour et installer des paquets

```
RUN apt-get update && apt-get install -y \  
    curl \  
    wget \  
    vim
```

Créer des répertoires

```
RUN mkdir -p /app/data
```

Commandes sur Dockerfile : RUN

Méthode 3 : Via Dockerfile

RUN - Exécution lors du build

dockerfile

```
FROM ubuntu:latest
```

```
# Exécuter une commande simple
```

```
RUN echo "GeeksForGeeks"
```

```
# Mettre à jour et installer des paquets
```

```
RUN apt-get update && apt-get install -y \  
    curl \  
    wget \  
    vim
```

```
# Créer des répertoires
```

```
RUN mkdir -p /app/data
```


Commandes sur Dockerfile : CMD

Méthode 3 : Via Dockerfile

CMD - Commande par défaut au démarrage

```
dockerfile
```

```
FROM ubuntu:latest
```

```
# Commande exécutée au démarrage du conteneur
```

```
CMD ["echo", "Bienvenue dans le conteneur!"]
```

Commandes sur Dockerfile : ENTRYPOINT

Méthode 3 : Via Dockerfile

ENTRYPOINT – Point d'entrée du conteneur

```
dockerfile
```

```
FROM ubuntu:latest
```

```
# Le conteneur exécutera toujours cette commande
```

```
ENTRYPOINT ["python", "app.py"]
```

Commandes sur Dockerfile : WORKDIR

Méthode 3 : Via Dockerfile

WORKDIR – Point d'entrée du conteneur

```
WORKDIR /app
```

```
WORKDIR /usr/src/application
```


Commandes sur Dockerfile : COPY/ADD

Méthode 3 : Via Dockerfile

COPY/ADD – Copie des fichiers locaux ou distants dans l'image

```
# COPY : fichiers locaux uniquement
```

```
COPY app.py /app/
```

```
COPY requirements.txt .
```

```
# ADD : fichiers locaux, archives tar et URLs
```

```
ADD project.tar.gz /opt/
```

```
ADD https://example.com/file.zip /tmp/
```

```
ADD run.sh /run.sh
```

Commandes sur Dockerfile : ENV

Méthode 3 : Via Dockerfile

ENV - Définit des variables d'environnement (persistantes dans le conteneur)

```
ENV NODE_ENV=production
```

```
ENV PORT=3000
```

```
ENV URL_POST=production.example.com
```

Commandes sur Dockerfile : Exemple complet (1/4)

Méthode 3 : Via Dockerfile

Exemple complet

1. Créer un fichier **Dockerfile**

```
FROM ubuntu:latest
```

```
# Mettre à jour les paquets  
RUN apt-get update
```

```
# Installer des outils  
RUN apt-get install -y curl wget
```

```
# Afficher un message (pendant le build)  
RUN echo "Image construite avec succès!"
```

```
# Créer un répertoire  
RUN mkdir /app
```

```
# Définir le répertoire de travail  
WORKDIR /app
```

```
# Copier un fichier  
COPY script.sh /app/
```

```
# Rendre le script exécutable  
RUN chmod +x /app/script.sh
```

```
# Commande par défaut au démarrage  
CMD ["bash"]
```

Commandes sur Dockerfile : Exemple complet (2/4)

Méthode 3 : Via Dockerfile

Exemple complet

1. Créer un fichier **Dockerfile**

2. Construire l'image

```
sudo docker build -t mon-image-custom .
```

```
FROM ubuntu:latest
```

```
# Mettre à jour les paquets  
RUN apt-get update
```

```
# Installer des outils  
RUN apt-get install -y curl wget
```

```
# Afficher un message (pendant le build)  
RUN echo "Image construite avec succès!"
```

```
# Créer un répertoire  
RUN mkdir /app
```

```
# Définir le répertoire de travail  
WORKDIR /app
```

```
# Copier un fichier  
COPY script.sh /app/
```

```
# Rendre le script exécutable  
RUN chmod +x /app/script.sh
```

```
# Commande par défaut au démarrage  
CMD ["bash"]
```

Commandes sur Dockerfile : Exemple complet (3/4)

Méthode 3 : Via Dockerfile

Exemple complet

1. Créer un fichier **Dockerfile**

2. Construire l'image

```
sudo docker build -t mon-image-custom .
```

3. Sortie attendue

```
Step 1/8 : FROM ubuntu:latest
Step 2/8 : RUN apt-get update
Step 3/8 : RUN apt-get install -y curl wget
Step 4/8 : RUN echo "Image construite avec succès!"
---> Running in abc123...
Image construite avec succès!
...
```

```
FROM ubuntu:latest
```

```
# Mettre à jour les paquets
RUN apt-get update
```

```
# Installer des outils
RUN apt-get install -y curl wget
```

```
# Afficher un message (pendant le build)
RUN echo "Image construite avec succès!"
```

```
# Créer un répertoire
RUN mkdir /app
```

```
# Définir le répertoire de travail
WORKDIR /app
```

```
# Copier un fichier
COPY script.sh /app/
```

```
# Rendre le script exécutable
RUN chmod +x /app/script.sh
```

```
# Commande par défaut au démarrage
CMD ["bash"]
```


Commandes sur Dockerfile : Exemple complet (4/4)

Méthode 3 : Via Dockerfile

Exemple complet

1. Créer un fichier **Dockerfile**

2. Construire l'image

```
sudo docker build -t mon-image-custom .
```

3. Sortie attendue

```
Step 1/8 : FROM ubuntu:latest
Step 2/8 : RUN apt-get update
Step 3/8 : RUN apt-get install -y curl wget
Step 4/8 : RUN echo "Image construite avec succès!"
---> Running in abc123...
Image construite avec succès!
...
```

4. Démarrer un conteneur depuis cette image :

```
docker run -it mon-image-custom
```

```
FROM ubuntu:latest
```

```
# Mettre à jour les paquets
RUN apt-get update
```

```
# Installer des outils
RUN apt-get install -y curl wget
```

```
# Afficher un message (pendant le build)
RUN echo "Image construite avec succès!"
```

```
# Créer un répertoire
RUN mkdir /app
```

```
# Définir le répertoire de travail
WORKDIR /app
```

```
# Copier un fichier
COPY script.sh /app/
```

```
# Rendre le script exécutable
RUN chmod +x /app/script.sh
```

```
# Commande par défaut au démarrage
CMD ["bash"]
```

